

The Natural Language Interaction Protocol and Standard for AI Agents

Luyi Xing¹, Rasit Onur Topaloglu², Ranjan Sinha³, Abhay Ratnaparkhi⁴, Samuel Ndichu⁵,
Christopher Nguyen⁶, Anindita Das³, Tom Sheffler⁷, Mohamed Rahouti⁸, Zichuan Li¹, Xiaojing
Liao¹, Sanjay Aiyagari⁹

¹University of Illinois at Urbana-Champaign; ²Marist University; ³IBM; ⁴eBay; ⁵NICT, Japan; ⁶Aitomatic, Inc.; ⁷Lenovo;
⁸Fordham University; ⁹Red Hat

{lxing2,zichuan7,xjliao}@illinois.edu, rasit.topaloglu@marist.edu, {rsinha,anindita.das}@us.ibm.com,
abratnaparkhi@ebay.com, ndichu@nict.go.jp, ctn@aitomatic.com, tsheffler@lenovo.com, mrahouti@fordham.edu,
sanjay@redhat.com

Abstract

AI agents are increasingly being developed and deployed across organizations using heterogeneous agent-development frameworks, AI models, tool interfaces, protocols, and execution environments. To realize their potential social and business impact, these agents must be able to interoperate through a common communication protocol. The Natural Language Interaction Protocol (NLIP), developed by researchers and practitioners across companies and universities and standardized by Ecma International, addresses this need by defining a standards-based application-layer protocol for AI-agent interaction. NLIP provides a lightweight semantic message envelope that can be carried over existing transports such as HTTP/HTTPS, WebSocket, and AMQP, while allowing NLIP-aware agents and gateways to adapt between clients, agents, local context stores, ontologies, tools, enterprise services, and heterogeneous underlying protocols. This paper presents the motivation and design rationale of NLIP, its message model and transport bindings, security-by-design considerations, reference implementation, representative applications, adoption signals, and relationship to emerging agent protocols such as MCP and A2A.

1 Introduction

AI-based software systems, or agents, are beginning to be deployed in a wide range of industrial applications. To realize their full potential, agents serving different functionalities and operated by different organizations must interoperate through a common communication protocol. Motivated by this need, a group of academic and industrial researchers collaborated to develop the *Natural Language Interaction Protocol (NLIP)*. NLIP was formally standardized in December 2025 by Ecma International Technical Committee 56 (TC56) as ECMA-430, together with related transport bindings and security-profile specifications [16–18, 20, 21]. In addition to Ecma International, the work leading to the standard has been supported by the Enterprise Neurosystem Group (enterpriseneurosystem.org) and the AI Alliance (thealliance.ai), two open-source organizations promoting collaboration across the broader AI community. This paper introduces the NLIP protocol, its key design innovations, security considerations, reference implementation, adoption signals, representative applications, and relation with relevant protocols.

Rather than replacing existing agent frameworks, tool protocols, or enterprise services, NLIP defines a standardized interaction layer that is independent of agent-internal technologies, logic, and

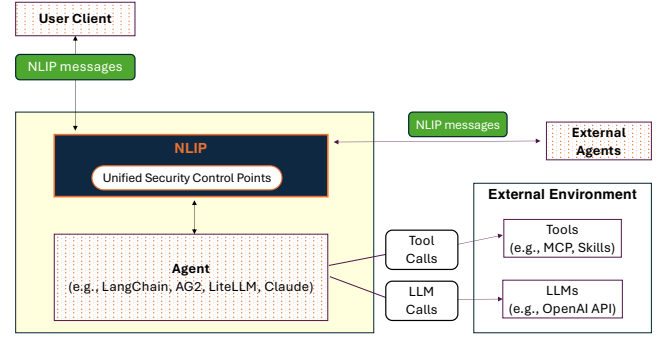


Figure 1: NLIP as an application-layer interoperability layer connecting user clients, agents, tools, LLMs, and external agents through standardized NLIP messages.

AI models being used by the agents. NLIP is an application-layer protocol: it standardizes agent interaction semantics and message envelopes while directly leveraging existing underlying transport protocols, including HTTP/HTTPS, WebSocket, and AMQP. NLIP-aware agents and gateways expose a common interface to user clients and other agents while continuing to use their native frameworks, LLM APIs, tool protocols, and enterprise services internally. As outlined in Figure 1, lightweight NLIP adapters enable heterogeneous agents to communicate through a common protocol without modifying their internal reasoning or execution logic. This architecture positions NLIP as a standards-based interoperability layer for heterogeneous agent ecosystems, enabling independently developed agents to communicate while preserving existing investments in frameworks, tools, and enterprise infrastructure.

In the following, § 2 describes the core design innovation of NLIP. § 3 provides an overview of the NLIP protocol design, including its message model and transport bindings. § 4 discusses security by design for agent communication. § 5 presents the reference implementation, development framework adoption, and representative applications. § 6 positions NLIP within the broader landscape of agent protocols and reports an empirical comparison with A2A (§ 6.1). § 7 concludes. The Appendix provides additional details on the origins and standardization history of NLIP.

2 Design Innovations of NLIP

Traditional application-layer protocols typically define fixed structures exchanged between communicating parties, such as JSON

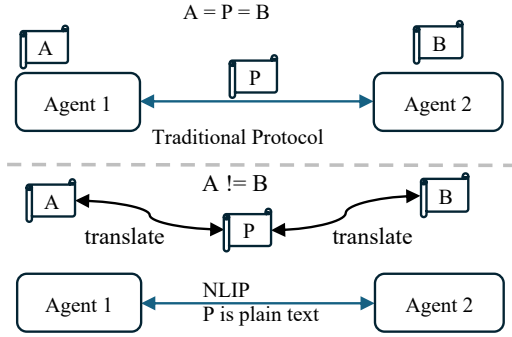


Figure 2: The distinctive design of NLIP

messages, web forms, or XML documents. This approach works well when clients and servers share a stable data model, but it becomes brittle in heterogeneous agent ecosystems, where agents, tools, models, ontologies, and task representations may evolve independently. Even modest changes to one endpoint’s internal representation can break communication unless schemas, versions, and compatibility rules are carefully managed. This tight coupling is a fundamental source of API-management complexity in distributed service environments and becomes especially limiting for AI-agent systems that must interoperate across organizations, frameworks, and application domains.

NLIP takes a different approach, which decouples the internal structure representation between the client and the server. Instead of specifying a rigid structure on the wire, natural language is used to transport the semantics of the information that the client and server need to exchange. Each side, the client and the server, can use an internal AI model to translate the natural language to its internal representation. This allows the client and server to maintain a different internal representation, if needed [20]. Because modern AI models are increasingly effective at translating between structured and unstructured versions of information, NLIP allows the definition of an application-level protocol that is more flexible and easier to maintain. This fundamental difference is shown in Figure 2.

In practice, this translation can be performed by NLIP-aware agents or gateways. These endpoints form the adaptive layer: they receive NLIP messages, ground them in local context, and translate them into the internal representations, tools, APIs, ontologies, or protocols used by the receiving system.

When the client and server need to exchange modalities different from text, e.g., video or audio, a similar paradigm can be followed. NLIP provides an envelope protocol that describes the type of content being exchanged, text, audio, video, or another modality, and lets the endpoints translate it to their internal representation.

3 NLIP Design Overview

Message model. To enable a multi-modal, flexible interaction among two parties involved in a communication, NLIP exchanges messages as JSON objects [20]. The message consists primarily of five fields, of which three are required, and two are optional. The three required fields include a content field, which contains the actual information being exchanged, a format field, which describes the format of the content, and a subformat field, which defines a

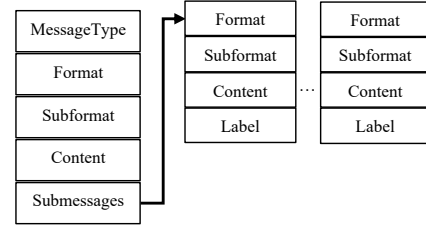


Figure 3: Structure of an NLIP message

further refinement of the format field. As an example, the content exchanged may be general text, the format would specify it as text, and the subformat would specify the language of the content, e.g., English. The optional fields include a message-type designation and an array of submessages. The message-type field can contain an application-specific string that can help the other side in parsing the content, while the submessage field allows several other pieces of content to be carried in the same message. The structure is shown in Figure 3.

Each entry in the array of submessages contains the three required fields of format, subformat, and content, and it contains an optional label field. The label, like the message type, can be used as hints to allow communicating parties to define their own conventions for rapid processing of content. As an example, the label field may indicate whether a submessage identifying location information is the pick-up location or the drop-off location for a journey.

The NLIP envelope is intentionally lightweight: it standardizes the interaction boundary without prescribing the internal ontology, reasoning method, model, or execution framework of each endpoint. This allows NLIP-aware agents to carry natural-language intent, structured context, provenance, constraints, and references to domain entities while preserving local implementation choices.

The format field may identify that the content being exchanged is natural text, with the subformat identifying the language. This is expected to be the predominant mode of information exchange. Other values of format fields include identifying a content as token (allows multiple related messages in a communication stream to be correlated) or an authentication token (to allow for authentication of communicating parties); indicating that the content is a structured text with the subformat identifying whether it is XML, JSON, HTML or something else; indicating that the content is binary data with the subformat identifying whether it is an audio with a specific encoding, video data with a specific encoding, sensor data etc.; marking a content as a location with the subformat identifying if the location is textual description or latitude/longitude description; and a generic format which can allow for arbitrary information to be processed by the other side.

Transport bindings. As an application-layer protocol, NLIP does not define a new network transport. Instead, it defines a lightweight message envelope and interaction semantics that can be carried over existing transport protocols. The current NLIP standards define official bindings over HTTP/HTTPS [17], WebSocket [18], and AMQP [16]. This design allows NLIP endpoints to reuse widely deployed transport infrastructure while providing a common semantic interaction layer for clients, agents, and gateways.

4 Security by Design for Agent Communication

Security profiles. NLIP defines security profiles for deployments with different assurance requirements [21]. However, AI-agent communication introduces risks beyond traditional transport security, including prompt injection, indirect prompt injection, unsafe tool invocation, sensitive-data leakage, and loss of provenance across agent chains [3, 33, 40]. NLIP defines three types of security profiles that should be supported by NLIP-based systems [21]. These profiles range from basic security requirements to more rigorous enterprise-oriented requirements intended to minimize common security risks in NLIP-based deployments. The rigorous security profile is designed to address the requirements of enterprise agents, which are often not addressed in commonly available standards or technical publications.

Unified control points for AI and agent security. Beyond defining security profiles, NLIP’s standardized communication layer provides a natural foundation for security enforcement across heterogeneous agent ecosystems. Because all communication between user clients and agents, and among agents themselves, is carried through standardized NLIP messages, validation, authorization, provenance tracking, policy enforcement, auditing, and security monitoring can be applied consistently at the communication layer rather than being fragmented across individual agent frameworks or applications. This provides a common foundation for mitigating both *AI-driven vulnerabilities*—such as prompt injection, indirect prompt injection, chain-of-thought leakage, semantic manipulation, and privacy violations [3, 25, 28, 33, 39–41]—and *system-level vulnerabilities*—such as session hijacking, confused-deputy and token-pass-through attacks, unauthenticated inference flooding, memory injection attacks, and data-at-rest exposure [2, 13–15, 26, 34–36, 42]. By providing a unified enforcement point for agent communication (Figure 1), NLIP enables consistent protection across independently developed agent frameworks while preserving interoperability.

5 Implementation and Example Applications

The flexibility and ease of use of NLIP have been explored through several proof-of-concept implementations at participating organizations. These examples illustrate a common architectural pattern: NLIP-aware agents act as adaptive interfaces between a standardized NLIP interaction layer and heterogeneous underlying systems, including tool protocols, enterprise APIs, knowledge bases, domain ontologies, industrial standards, and vendor-specific services. Several of these use cases have been described in the technical report [19] describing NLIP motivation. Those include the use of NLIP as a question-answering service, as a proxy federating across several agents, as a gateway in front of a service using MCP to invoke tools [19, 29], a speech-based customer-service agent, and as an integrator of services in a hybrid cloud environment (specifically integrating Active Directory with a ServiceNow System). In this pattern, NLIP does not replace MCP. Instead, an NLIP-aware agent or gateway exposes a standard NLIP interface to the client while using MCP internally to invoke tools and services. MCP remains the tool/service protocol, while NLIP provides the interaction envelope for agents and clients.

5.1 Reference Implementation of NLIP

The NLIP protocol is accompanied by open-source reference implementations and software development kits (SDKs), including `nlip_sdk`, `nlip_client`, and `nlip_server` [31]. Together, these libraries implement the core NLIP protocol, including message structures, transport bindings, serialization, and client/server communication primitives. The SDKs are intentionally independent of agent-internal application logic and development frameworks, allowing existing agents and applications to adopt NLIP without modifying their internal execution models. This separation enables developers to focus on application logic while relying on reusable, standards-compliant implementations for agent communication, facilitating interoperability across heterogeneous AI-agent ecosystems.

5.2 Extension to Agent Development Frameworks and Adoption Signals

The framework-independent architecture of NLIP enables existing agent systems to expose standardized NLIP interfaces through lightweight adapters, without changing their internal reasoning, planning, or tool-invocation logic. Agents developed using different frameworks can therefore communicate with user clients and other NLIP-compatible agents through a common protocol.

This extensibility has been demonstrated through the integration of NLIP into the AG2 [1] agent development framework in April 2026 [4, 5]. AG2 reports more than one million monthly downloads (1,087,122 downloads in March 2026) and over 57,000 daily downloads with peaks exceeding 68,000, and is widely used in industry and research across diverse application domains. As AG2 adds support for NLIP, these deployments provide early evidence of NLIP’s growing footprint as a standardized interaction layer for large-scale agent systems.

5.3 Examples of NLIP Applications

Self-healing of Telecom equipment. A commercial use case was demonstrated by IBM at the 2025 Telecommunications Management Forum (TMF) [38], featuring a collection of intelligent AI agents that would automatically diagnose the root cause of an equipment failure and (subject to applicable policies) automatically reboot or reconfigure the device at fault. The communication with the device was conducted using standards specified by TMF, with NLIP-based AI agents invoking those standards to monitor and reconfigure the devices. The system consisted of two Agents that leveraged the IBM Granite Mixture of Expert models to diagnose and fix the problem, one identifying the root cause (the Event Agent) and the other identifying the actions to be followed to inform the customer of any problem using a TMF Forum API. NLIP was used by the two intelligent agents to communicate with each other.

As the system was being prepared for demonstration, a useful attribute of NLIP was discovered. In order to demonstrate the operations of the agents, a demonstration console was needed. While the typical demonstration console would have required supporting a special interface to get demonstration information from the different systems involved in this demonstration, NLIP provided a convenient interface to put a demonstration console above the system being developed. The existing component, namely the event collector, required a special interface to show what it was executing,

but the NLIP-based agents could be demonstrated without requiring any additional protocol support.

Multimodal NLIP agent. In this use case [19], three specialized agents, Speech to Text Agent, Channel Recommender Agent, and Search Agent, collaborate using NLIP to fulfill a customer’s voice-based support request in real time. Each agent plays a distinct role in the end-to-end interaction pipeline, showcasing how modular AI components can be composed using standardized communication and interoperable APIs. The spoken audio is transcribed into text using a speech-to-text model. Based on keywords extracted from the customer’s request, the system identifies relevant Reddit channels. A specialized search agent then queries these channels and retrieves the most recent and relevant posts, delivering them back to the customer in real time.

This transcribed content is then processed by a channel recommender agent, which may use an LLM enabling contextual understanding and intent extraction. Additionally, the search agent integrates MCP with Reddit’s search APIs to retrieve relevant, real-time information. This is an example of NLIP being used on the client- and agent-facing side for semantic interaction, while MCP is used internally to access tools and APIs. This use case illustrates how multi-agent systems can dynamically support customer queries across external knowledge sources.

6 NLIP and Related Protocols

NLIP belongs to a broader family of agent communication efforts, from earlier agent communication languages such as the Knowledge Query and Manipulation Language (KQML) [22] and the Foundation for Intelligent Physical Agents Agent Communication Language (FIPA ACL) [23, 24] to recent protocols such as the Model Context Protocol (MCP) [29], Agent2Agent (A2A) [9], the Agent Communication Protocol (ACP) [6], and the Agent Network Protocol (ANP) [12]. Other emerging efforts address adjacent parts of the agent ecosystem, including Media over QUIC Transport (MoQ)-based proposals for low-latency agent transport and trust-oriented protocols such as the Agent Trust Protocol (ATP) for agent identity, authorization, and attestation [27, 30, 32]. Public NLIP project materials were available in 2024 [31], followed by an overview paper presented in March 2025 at the AAAI ’25 Workshop on Open-Source AI for Mainstream Use [10].

NLIP is best understood as a neutral application-layer semantic interaction protocol that complements, rather than replaces, existing agent, tool, and enterprise protocols. In Mode A, an NLIP-aware agent or gateway exposes an NLIP-facing interface to clients and peer agents, while using protocols such as MCP, A2A, ACP, enterprise APIs, or industrial protocols internally or toward underlying services. In Mode B, NLIP-aware agents mediate interoperability among agents implemented with different underlying protocols. This allows NLIP to serve as a semantic coordination layer across a fragmented protocol ecosystem, while preserving existing investments in tool protocols, agent frameworks, and enterprise infrastructure.

An analogy to the Web. At the agent-ecosystem level, NLIP is designed to play a role analogous to HTTP/HTTPS on the Web: a common interaction layer that allows independently implemented endpoints to communicate through standardized messages. Just

as Web browsers and servers exchange HTTP/HTTPS messages regardless of the application semantics carried above them, AI agents can exchange NLIP messages regardless of their internal frameworks, task structures, workflow patterns, or execution environments. Higher-level semantic abstractions, such as A2A tasks and agent cards, can naturally be carried within NLIP messages, much like HTML documents, REST APIs, or application-specific data formats are transported over HTTP/HTTPS. This layering makes NLIP suitable as a general interoperability substrate, while allowing more specialized protocols to define higher-level workflow semantics above or alongside it.

6.1 An Empirical Comparison of NLIP and A2A

Design goals. At a high level, both NLIP and A2A support agent-to-agent communication, but they make different design choices. A2A originated from Google and follows a task-oriented client-server model in which a client agent delegates work to a remote agent [11]. Its protocol model includes predefined structures such as tasks, task states including “input-required” [8], and agent cards with fixed fields [7]. These abstractions support structured task delegation, but also impose a specific task model. In contrast, NLIP does not hard-code tasks, task states, workflows, or delegation relationships into its core message model; agents define interaction semantics through NLIP messages, enabling arbitrary conversations, workflows, and coordination patterns across heterogeneous systems.

Performance overhead. A2A’s richer predefined structures can support governance-intensive workflows, but may add runtime overhead in connection handling, message exchange, serialization, and task-state management. In a controlled comparison using the same proof-of-concept three-agent customer-support workflow, NLIP achieved 7.6–9.6× lower end-to-end latency than A2A-SDK across repeated runs [37]. Message creation overhead was negligible, while A2A-SDK connection establishment dominated the gap; with connection caching enabled, NLIP still retained a 2.3–2.8× advantage because overhead shifted to the send phase rather than disappearing [37]. Against Python-A2A, NLIP maintained a 3.1–4.3× advantage [37]. These results suggest that NLIP is well suited for frequent, lightweight, latency-sensitive interactions, while A2A-style task abstractions are better suited for long-running or governance-intensive workflows and can be carried over, integrated with, or bridged through NLIP.

7 Conclusion

NLIP is a formal standard specifically designed for natural-language-centered interaction among modern AI agents, and broader adoption is emerging. NLIP’s broader significance is that it standardizes the semantic envelope for interaction, while NLIP-aware agents and gateways provide the adaptive glue across heterogeneous, domain-specific agent ecosystems. This makes NLIP a candidate interoperability layer for multi-protocol, ontology-rich, and locally controlled AI deployments.

References

- [1] [n. d.]. AG2 (formerly AutoGen). <https://github.com/ag2ai/ag2>.
- [2] 2025. Model Context Protocol (MCP) Security Best Practices, Version 2025-11-25. https://modelcontextprotocol.io/specification/2025-11-25/basic/security_best_practices.

- [3] Sahar Abdelnabi, Kai Greshake, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISeC '23)*. ACM, 79–90. doi:10.1145/3605764.3623985
- [4] AG2 Contributors. 2026. *Add Natural Language Interaction Protocol (NLIP) Support*. GitHub Pull Request #2468. <https://github.com/ag2ai/ag2/pull/2468>
- [5] AG2 Contributors. 2026. *Enhancements to Natural Language Interaction Protocol (NLIP) Integration*. GitHub Pull Request #3024. <https://github.com/ag2ai/ag2/pull/3024>
- [6] Agent Communication Protocol Project. n.d.. *Agent Communication Protocol (ACP) Documentation*. <https://agentcommunicationprotocol.dev/>
- [7] Agent2Agent Protocol Project. 2026. *Agent Card*. <https://agent2agent.info/docs/concepts/agentcard/>
- [8] Agent2Agent Protocol Project. 2026. *Task*. <https://agent2agent.info/docs/concepts/task/>
- [9] Agent2Agent Protocol Project. n.d.. *Agent2Agent (A2A) Protocol Specification*. <https://a2a-protocol.org/latest/specification/>
- [10] Sanjay Aiyagari et al. 2025. An Overview of the Natural Language Interaction Protocol. In *AAAI 2025 Workshop on Open-Source AI for Mainstream Use*. <https://www.eecis.udel.edu/~mlm/docs/2025-Aiyagari-AAAI-25-AnOverviewOfTheNaturalLanguageInteractionProtocol-Workshop.pdf>
- [11] Alan Blount, Frank Guan, and Nick Losier. 2026. *How A2A Is Building a World of Collaborative Agents*. Google Developers Blog. <https://developers.googleblog.com/how-a2a-is-building-a-world-of-collaborative-agents/>
- [12] Gaowei Chang, Eidan Lin, Chengxuan Yuan, Rizhao Cai, Binbin Chen, Xuan Xie, and Yin Zhang. 2025. Agent Network Protocol Technical White Paper. *arXiv preprint arXiv:2508.00007* (2025). doi:10.48550/arXiv.2508.00007
- [13] Jay Chen, Royce Lu, and Palo Alto Networks Unit 42. 2025. When AI Agents Go Rogue: Agent Session Smuggling Attack in A2A Systems. <https://unit42.paloaltonetworks.com/agent-session-smuggling-in-agent2agent-systems/>
- [14] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. Agent-poison: Red-teaming llm agents via poisoning memory or knowledge bases. *Advances in Neural Information Processing Systems* 37 (2024), 130185–130213.
- [15] Jian Cui, Zichuan Li, Luyi Xing, and Xiaojing Liao. 2025. Safeguard-by-development: A privacy-enhanced development paradigm for multi-agent collaboration systems. *arXiv preprint arXiv:2505.04799* (2025).
- [16] Ecma International. 2025. Binding of the Natural Language Interaction Protocol (NLIP) over AMQP.
- [17] Ecma International. 2025. Binding of the Natural Language Interaction Protocol (NLIP) over HTTP/HTTPS.
- [18] Ecma International. 2025. Binding of the Natural Language Interaction Protocol (NLIP) over WebSocket.
- [19] Ecma International. 2025. *Explanatory Guide to the Natural Language Interaction Protocol (NLIP)* (1 ed.). Report ECMA TR/113.
- [20] Ecma International. 2025. Natural Language Interaction Protocol (NLIP).
- [21] Ecma International. 2025. Security Profiles for Natural Language Interaction Protocol (NLIP).
- [22] Tim Finin, Richard Fritzson, Donald P. McKay, and Robin McEntire. 1994. KQML as an Agent Communication Language. In *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM '94)* (Gaithersburg, MD, USA). ACM, 456–463. doi:10.1145/191246.191322
- [23] Foundation for Intelligent Physical Agents. 2002. FIPA ACL Message Structure Specification. <http://www.fipa.org/specs/fipa00061/SC00061G.html>
- [24] Foundation for Intelligent Physical Agents. 2002. FIPA Communicative Act Library Specification. <http://www.fipa.org/specs/fipa00037/SC00037J.html>
- [25] Tommaso Green, Martin Gubri, Haritz Puerto, Sangdoo Yun, and Seong Joon Oh. 2025. Leaky Thoughts: Large Reasoning Models Are Not Private Thinkers. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [26] Abhinav Kumar, Jaechul Roh, Ali Naseh, Marzena Karpinska, Mohit Iyyer, Amir Houmansadr, and Eugene Bagdasarian. 2025. Overthink: Slowdown attacks on reasoning llms. *arXiv preprint arXiv:2502.02542* (2025).
- [27] D. Liu and S. Krishnan. 2026. *Agent Protocol over MoQ*. Internet-Draft draft-liu-agent-protocol-over-moq-00. IETF. <https://www.ietf.org/archive/id/draft-liu-agent-protocol-over-moq-00.html>
- [28] Matthieu Meeus, Shubham Jain, Marek Rei, and Yves-Alexandre de Montjoye. 2024. Did the Neurons Read your Book? Document-level Membership Inference for Large Language Models. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*.
- [29] Model Context Protocol Contributors. 2025. *Model Context Protocol Specification, version 2025-11-25*. <https://modelcontextprotocol.io/specification/2025-11-25>
- [30] S. Nandakumar and C. Jennings. 2026. *MOQ Transport for Agent Protocols*. Internet-Draft draft-nandakumar-ai-agent-moq-transport-00. IETF. <https://www.ietf.org/archive/id/draft-nandakumar-ai-agent-moq-transport-00.html>
- [31] NLIP Project Contributors. 2024. *NLIP Project Documents*. <https://github.com/nlip-project/documents>
- [32] OTT Cybersecurity LLC. 2026. *Agent Trust Protocol (ATP)*. <https://github.com/OTT-Cybersecurity-LLC/lyrie-ai/blob/main/packages/atp/README.md>
- [33] OWASP Foundation. 2025. *OWASP Top 10 for Large Language Model Applications: LLM01:2025 Prompt Injection*. <https://genai.owasp.org/llm-top-10/>
- [34] OWASP GenAI Security Project. 2025. LLM10:2025 Unbounded Consumption. <https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/>
- [35] Ayush RoyChowdhury, Mulong Luo, Prateek Sahu, Sarbartha Banerjee, and Mohit Tiwari. 2024. Confusedpilot: Confused deputy risks in rag-based llms. *arXiv preprint arXiv:2408.04870* (2024).
- [36] Avital Shafra, Roei Schuster, and Vitaly Shmatikov. 2025. Machine Against the {RAG}: Jamming {Retrieval-Augmented} Generation with Blocker Documents. In *34th USENIX Security Symposium (USENIX Security 25)*. 3787–3806.
- [37] Ranjan Sinha, Anindita Das, Ashika Anand Babu, and Hari Palleti. n.d.. Protocol-Level Efficiency in Agent Communication: An Empirical Comparison of NLIP and A2A. In *30th Annual CASIS Workshop*.
- [38] TM Forum. 2026. *TM Forum*. <https://www.tmforum.org/>
- [39] Shouju Wang, Fenglin Yu, Xirui Liu, Xiaoting Qin, Jue Zhang, Qingwei Lin, Dongmei Zhang, and Saravan Rajmohan. 2025. Privacy in action: Towards realistic privacy mitigation and evaluation for llm-powered agents. *arXiv preprint arXiv:2509.17488* (2025).
- [40] Qiushi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024* (Bangkok, Thailand). Association for Computational Linguistics, 10471–10506. doi:10.18653/v1/2024.findings-acl.624
- [41] Muyang Zheng, Yuanzhi Yao, Changting Lin, Rui Wang, and Meng Han. 2025. MIST: Jailbreaking Black-box Large Language Models via Iterative Semantic Tuning. *ArXiv abs/2506.16792* (2025).
- [42] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2025. {PoisonedRAG}: Knowledge corruption attacks to {Retrieval-Augmented} generation of large language models. In *34th USENIX Security Symposium (USENIX Security 25)*. 3827–3844.

Origins of NLIP

NLIP started in June 2024 as a technical working group within the Enterprise NeuroSystem Group (ENG). ENG is an open-source consortium involving 30+ companies and universities formed with the goal of enabling technologies needed to establish a global AI network. The original members of the working group included researchers from the University of Delaware, the University at Buffalo, the University of Michigan, SRI International, IBM, and Red Hat. The group eventually expanded to further include researchers from Cisco, Fordham University, Pennsylvania State University, and Indiana University Bloomington. In October 2024, the initial version of the protocol was defined, and a decision to convert the specification to a formal standard was made. Ecma International, the group that defines various application-level technologies such as JavaScript, was identified as the one closest to the needs of an application-level protocol design, and the group was formally inducted as a technical working group within ECMA in December 2024. The effort was joined by ServiceNow and Hitachi, which supported the formation of the group in ECMA. In early 2025, ENG became an official member of the AI Alliance, a consortium of several companies and universities involved in promoting open-source AI technologies and governance. The Alliance decided to incorporate the working group as part of its activities, providing resources such as a GitHub space for the group to continue its activities as well. Subsequently, researchers from several other organizations joined the effort, including those from Qualcomm, Japan National Institute of Information and Communications Technology (NICT), University of Illinois Urbana-Champaign (UIUC), Marist University, Chicago State University, City University of New York, Microsoft, and Palo Alto Networks.

Ecma requires formal membership of organizations in order to participate, while ENG allows open participation of any technical person from any organization. The members who officially joined Ecma TC56 to define the official standards included IBM, Hitachi, Red Hat, ServiceNow, Purdue University, Indiana University Bloomington, University at Buffalo, Fordham University, Chicago State University, Marist University, and City University of New York. Researchers at other organizations contributed only as technical advisors as part of the ENG working group. The ENG group

met weekly to discuss technical issues and implement prototypes, while the Ecma standard team met monthly to formalize the standards. This bi-pronged approach allowed us to make rapid technical progress while making steady progress towards the stage of official standardization.

The ENG effort also allowed the group to explore the design of NLIP through several proof-of-concept prototypes, which are described in the NLIP explanatory report and summarized later in this paper [19].